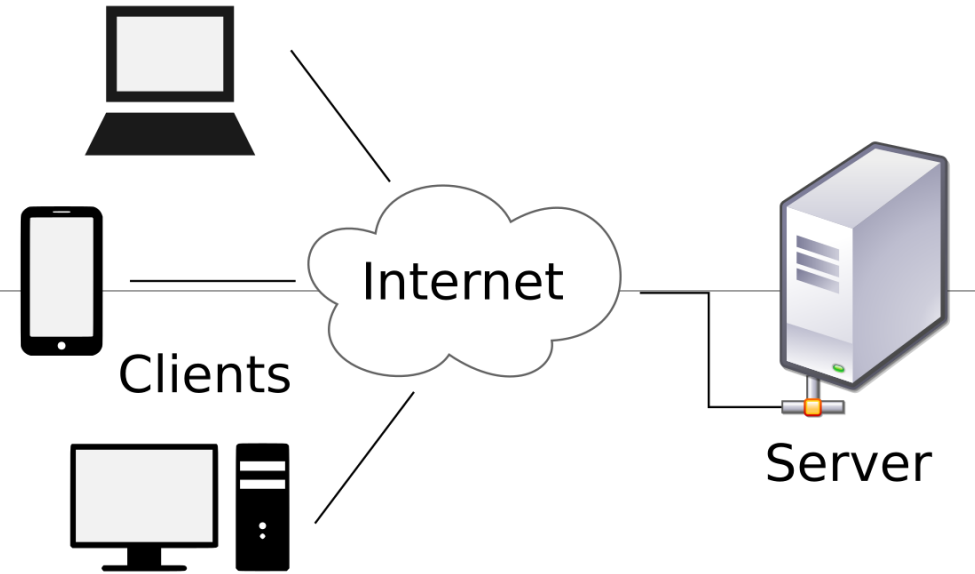


HTTP enhancements

SSE, HTTP2

HTTP is client driven



If client 2 updates important data that client 1 just pulled in, unless client 1 makes a second request for an update the server cannot notify client 1

Server is not able to push data as and when needed

Pushing Data

Manual- User may click on “refresh” when (s)he has decided to pull the data

Every time the window is opened data is pulled from the server

Periodic update

- Overhead on the server processes as it needs to process the request even when no update is to be notified
- Polling
- Exponential backoff



Server Data Push

```
id: 123
event: message
data: Hello World
data: This is another message line
};
```

```
evtSource.onmessage = (event) => {
    const newElement =
    document.createElement("li");
    const eventList =
    document.getElementById("list");
    newElement.textContent = `message:
                                ${event.data}`;
    eventList.appendChild(newElement);
};
```

Polling involves the client making repeated HTTP requests to the server, asking if new data is available. This is inefficient because the client must keep sending requests even if there's no new data.

Server Sent Event (SSE)-an API for managing and parsing events from long-running HTTP connections

With server-sent events, it's possible for a server to send new data to a web page at any time, by pushing messages to the web page. These incoming messages can be treated as **Events + data** inside the web page.

program the server to stream events to the front-end, but the client side code works almost identically to websockets in part of handling incoming events. This is a one-way connection, so you can't send events from a client to a server.

Messages sent from the server that don't have an event field are received as message events. To receive message events, attach a handler for the message event

Sending events

```
res.write(`data:messages\n\n`);
```

```
evtSource.onmessage = (event) => {...}
```

```
res.write(`event: customEvent\n`);  
res.write(`data: This is a custom event message\n\n`);
```

```
eventSource.addEventListener('customEvent', function(event) {  
  console.log('Custom event received:', event.data);  
});
```

Steps of SSE

The client makes a standard HTTP request to the server.

The server holds this connection open and sends updates to the client as needed.

The client listens for these updates and processes them in real-time.

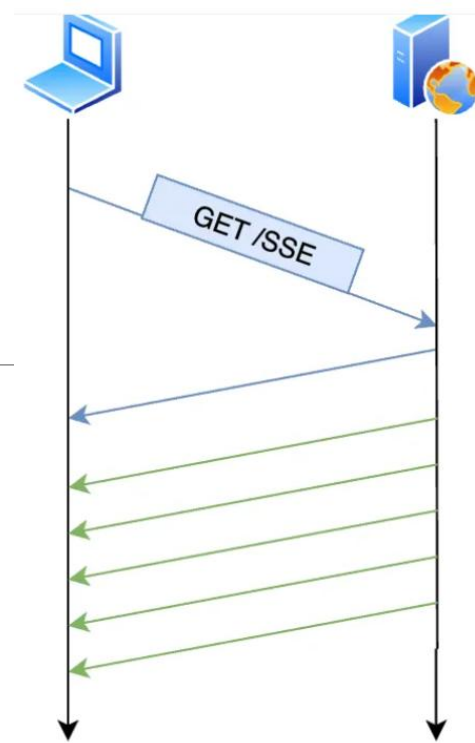
Applications: live news feeds, stock price updates where the client is not expected to send but receive messages

Simplicity, lightweight, cross-browser support

Automatic reconnection: The browser automatically reconnects to the server if the connection is dropped, making it robust and easy to maintain.

Head of line blocking problem

- if the client sends requests A, B, and C, but request A requires a lot of server resources, B and C are blocked until A finishes



Background

SPDY was an experimental protocol, developed at Google and announced in mid-2009, whose primary goal was to try to reduce the load latency of web pages by addressing some of the well-known performance limitations of HTTP/1.1

For example, 50% reduction in page load time

Fast-forward to 2012 and the new experimental protocol was supported in Chrome, Firefox, and Opera, and a rapidly growing number of sites, both large (e.g., Google, Twitter, Facebook) and small, were deploying SPDY within their infrastructure.

Observing the above trend, the HTTP Working Group (HTTP-WG) initiated a new effort to take the lessons learned from SPDY, build and improve on them, and deliver an official "HTTP/2" standard

In early 2015 the IESG reviewed and approved the new HTTP/2 standard for publication

Accordingly, SPDY was removed from the browsers

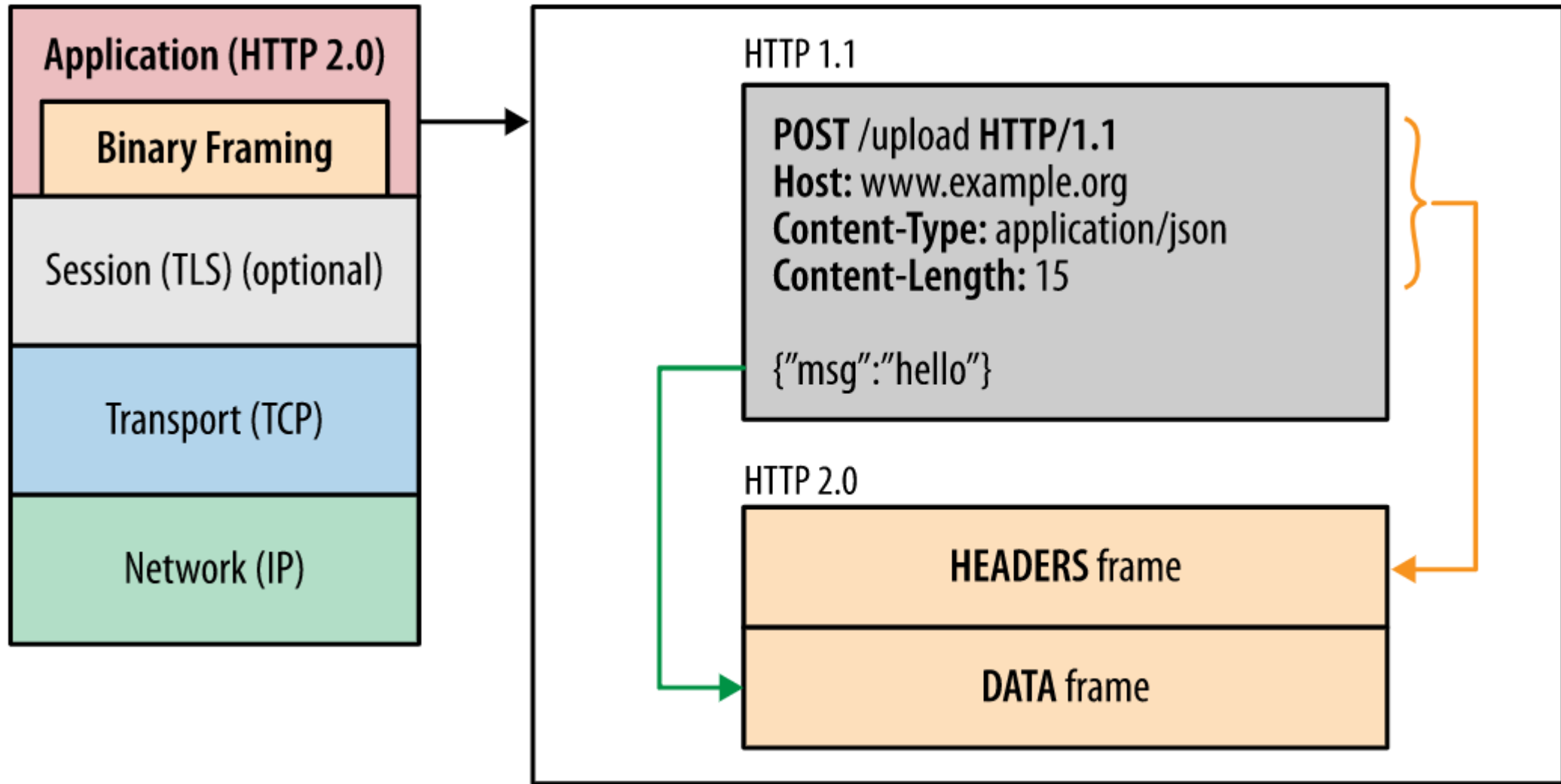
HTTP 1.x → 2.0

Problems of HTTP1.x

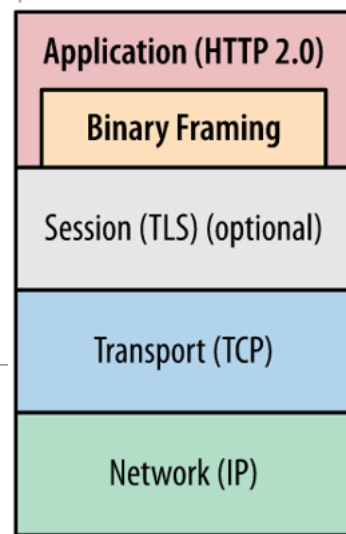
- ❑ HTTP/1.x clients need to use multiple connections to achieve concurrency and reduce latency;
- ❑ HTTP/1.x does not compress request and response headers, causing unnecessary network traffic;
- ❑ HTTP/1.x does not allow effective resource prioritization, resulting in poor use of the underlying TCP connection

HTTP 2.0

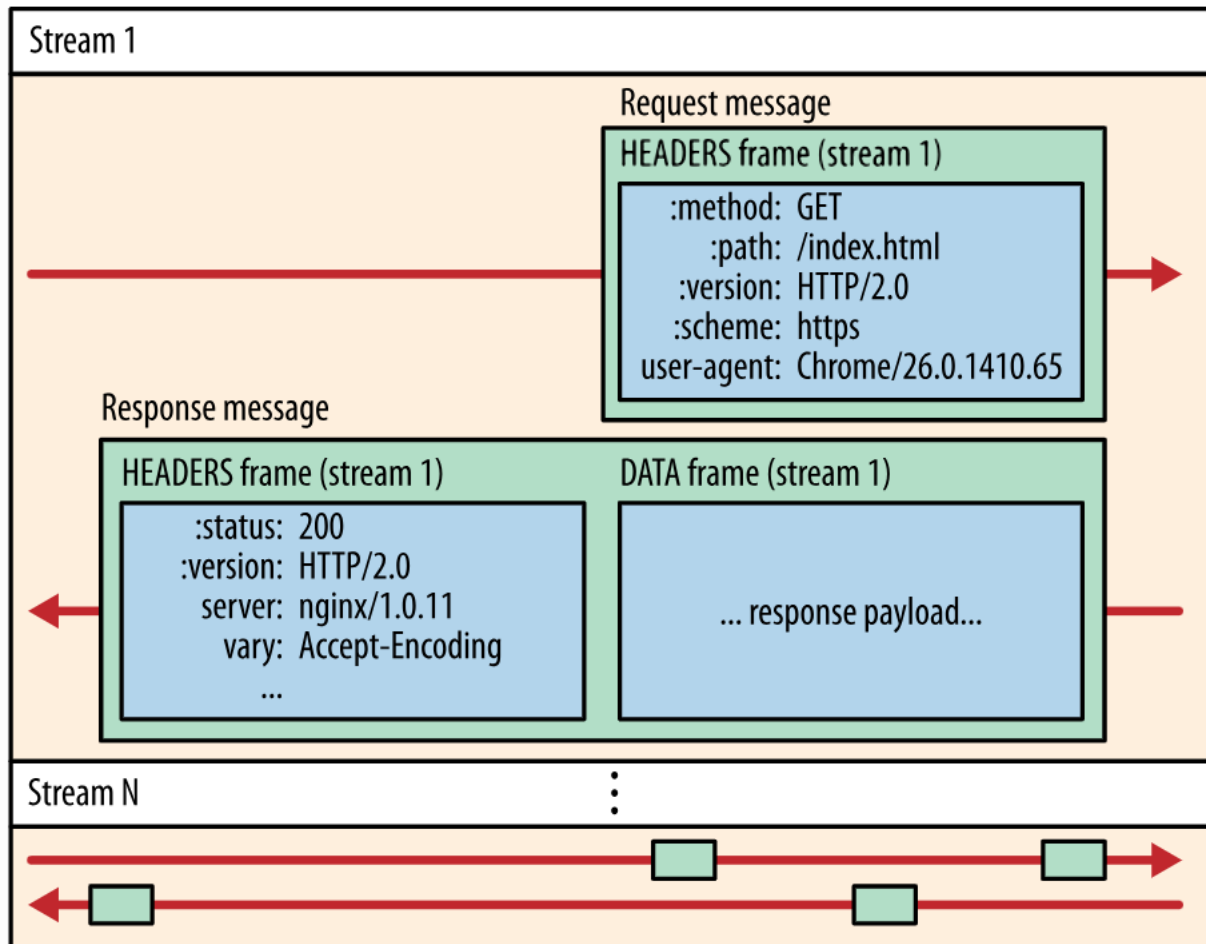
- ❑ The application semantics of HTTP are the same, and no changes were made to the offered functionality or core concepts such as HTTP methods, status codes, URIs, and header fields
- ❑ while the high-level API remains the same, it is important to understand how the low-level changes address the performance limitations of the previous protocols



Binary Framing Layer



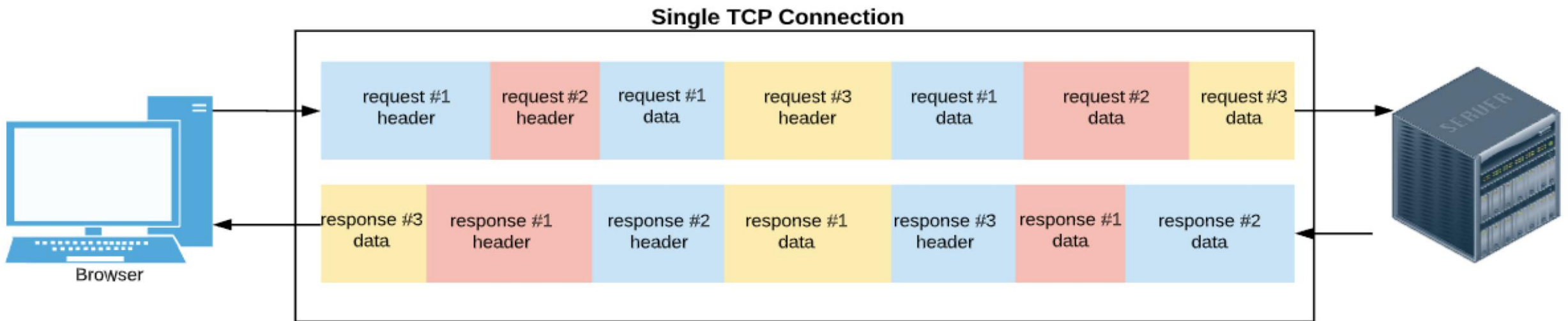
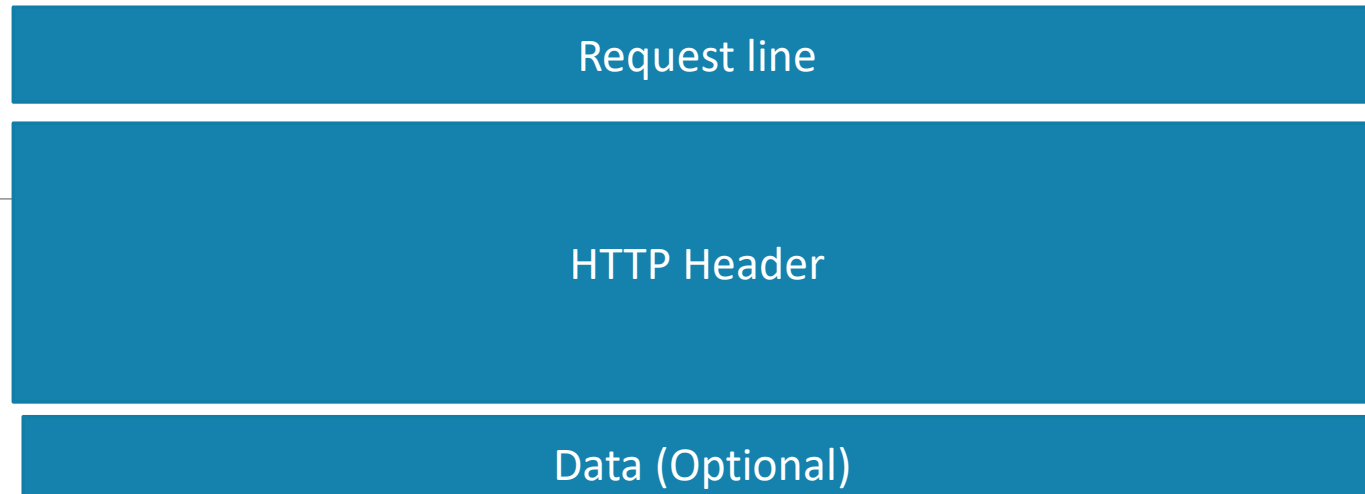
- ❑ The "layer" refers to a design choice to introduce a new optimized encoding mechanism between the socket interface and the higher HTTP API exposed to our applications: the HTTP semantics, such as verbs, methods, and headers, are unaffected, but the way they are encoded while in transit is what's different.
- ❑ Unlike the newline delimited plaintext HTTP/1.x protocol, all HTTP/2 communication is split into smaller messages and frames, each of which is encoded in binary format.
- ❑ As a result, both client and server must use the new binary encoding mechanism to understand each other: an HTTP/1.x client won't understand an HTTP/2 only server, and vice versa.
- ❑ Thankfully, our applications remain blissfully unaware of all these changes, as the client and server perform all the necessary framing work on our behalf.



- *Stream*: A bidirectional flow of bytes within an established connection, which may carry one or more messages.
- *Message*: A complete sequence of frames that map to a logical request or response message.
- *Frame*: The smallest unit of communication in HTTP/2, each containing a frame header, which at a minimum identifies the stream to which the frame belongs.

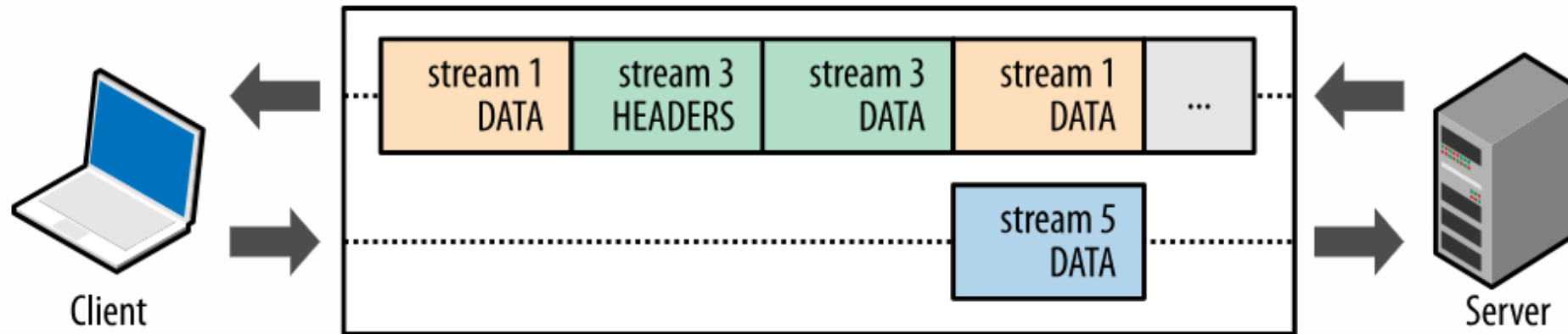
Each stream may be assigned an integer weight between 1 and 256.

Each stream may be given an explicit dependency on another stream.



http/2

HTTP/2 breaks down the HTTP protocol communication into an exchange of binary-encoded frames, which are then mapped to messages that belong to a particular stream, and all of which are multiplexed within a single TCP connection.



There are three parallel streams in flight! 1,3, and 5

Http/2 benefits

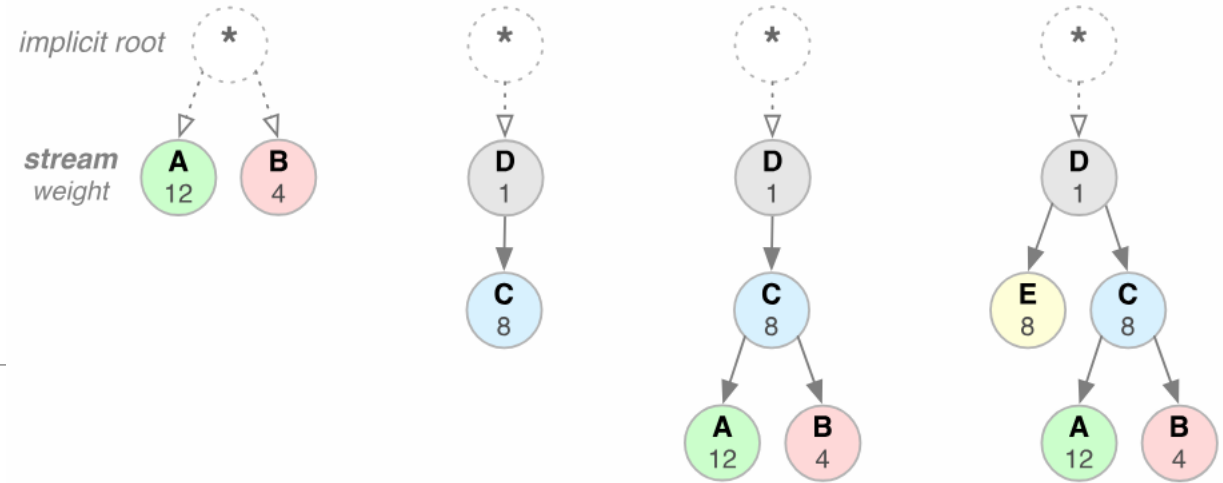
- With HTTP/1.x, if the client wants to make multiple parallel requests to improve performance, then multiple TCP connections must be used
- This also results in head-of-line blocking and inefficient use of the underlying TCP connection

The ability to break down an HTTP message into independent frames, interleave them, and then reassemble them on the other end is the single most important enhancement of HTTP/2.

In fact, it introduces a ripple effect of numerous performance benefits across the entire stack of all web technologies

- Interleave multiple requests in parallel without blocking on any one.
- Interleave multiple responses in parallel without blocking on any one.
- Use a single connection to deliver multiple requests and responses in parallel.
- Deliver lower page load times by eliminating unnecessary latency and improving utilization of available network capacity.

Stream prioritization



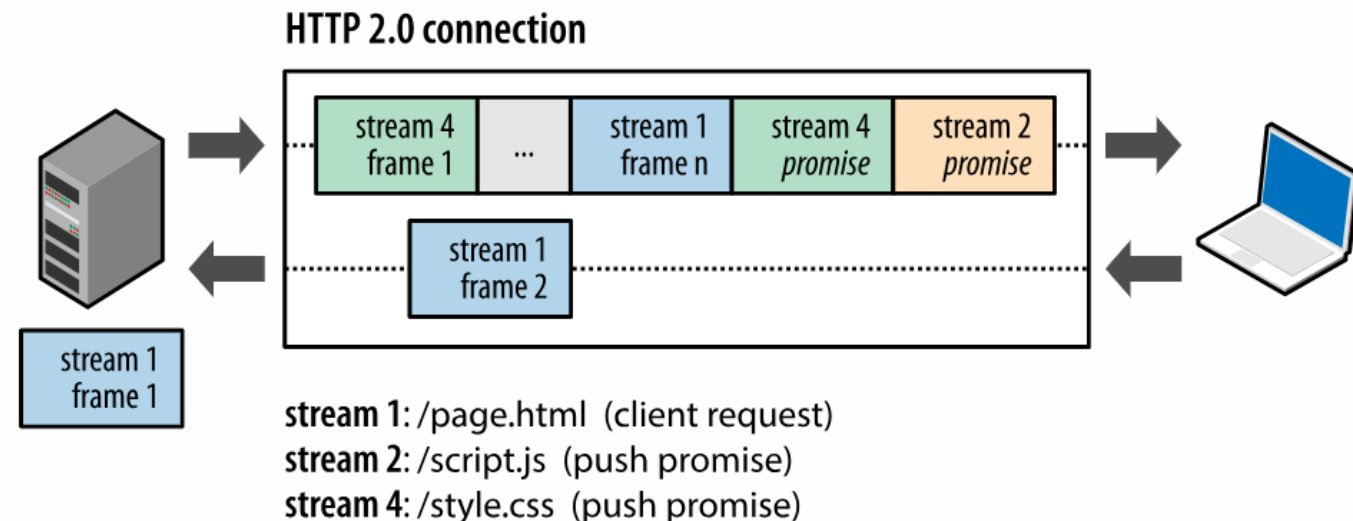
- ❑ A stream dependency within HTTP/2 is declared by referencing the unique identifier of another stream as its parent;
- ❑ if omitted the stream is said to be dependent on the "root stream".
- ❑ Declaring a stream dependency indicates that, if possible, the parent stream should be allocated resources ahead of its dependencies—e.g., please process and deliver response D before response C.
- ❑ Streams that share the same parent (i.e., sibling streams) should be allocated resources in proportion to their weight.
- ❑ For example, if stream A has a weight of 12 and its one sibling B has a weight of 4, then to determine the proportion of the resources that each of these streams should receive- $12/16$ and $4/16$

One connection only

- ❑ With the new binary framing mechanism in place, HTTP/2 no longer needs multiple TCP connections to multiplex streams in parallel;
- ❑ each stream is split into many frames, which can be interleaved and prioritized.
- ❑ As a result, all HTTP/2 connections are persistent, and only one connection per origin is required, which offers numerous performance benefits.
- ❑ Most HTTP transfers are short and bursty, whereas TCP is optimized for long-lived, bulk data transfers.
- ❑ By reusing the same connection HTTP/2 is able to both make more efficient use of each TCP connection, and also significantly reduce the overall protocol overhead.
- ❑ Further, the use of fewer connections reduces the memory and processing footprint along the full connection path

Server Push

- ❑ Another powerful new feature of HTTP/2 is the ability of the server to send multiple responses for a single client request. That is, in addition to the response to the original request, the server can *push* additional resources to the client, without the client having to request each one explicitly!
- ❑ A typical web application consists of dozens of resources, all of which are discovered by the client by examining the document provided by the server.
- ❑ As a result, why not eliminate the extra latency and let the server push the associated resources ahead of time? The server already knows which resources the client will require; that's server push.



Creating a server instance in Express

```
const server = http2.createSecureServer({  
  key: fs.readFileSync('localhost-privkey.pem'),  
  cert: fs.readFileSync('localhost-cert.pem'),  
});  
  
server.on('error', (err) => console.error(err));
```

On the server side, user code will typically not listen for stream event directly, and would instead register a handler for the 'stream' event emitted by the `net.Server` or `tls.Server` instances returned by `http2.createServer()` or `http2.createSecureServer()`

```
server.on('stream', (stream, headers) => {  
  stream.respond({  
    'content-type': 'text/html; charset=utf-8',  
    ':status': 200,  
  });  
  stream.write('hello ');  
  // Getting session object by using session method  
  const http2session = stream.session;  
  stream.on('error', (error) => console.error(error));  
  stream.end('<h1>Hello World</h1>');  
});
```

Even though HTTP/2 streams and network sockets are not in a 1:1 correspondence, a network error will destroy each individual stream and must be handled on the stream level

Cookies

They are very small limited pieces of data, that the server sends back to the client

Goes through response header

Size of the cookie has to be small

Date/lifetime

Encryption while sending the cookie

- client only sends this cookie back to the server if a secure link, or an HTTPS communication protocol is being used